Software Requirements Specification: Personal Information Management System

Course: COMP3211 Software Engineering, Fall 2026
Project: Command-line Personal Information Management (PIM) system
Group number: 89
Members and student IDs: [Human input pending: member names and student IDs]
Version: Independent-review candidate 1.3
Date: 2026-10-23
Implementation basis: Current candidate source and tests, including local review changes; historical starting commit `4d3508db0447a3ac6346a2a2789da89208f88fd5` does not identify the complete delivery. See the candidate manifest for packaged file hashes.

1. Preface

This specification is for the project group, course assessors, designers, implementers, and testers. It states the externally observable PIM behavior used to assess the current candidate implementation. The official 2026 Course Project Description, especially Appendix B (p. 4), supplies the required stories. Lecture 04, PDF pp. 26–27 and 31–32, supplies the writing guidance and document structure. The concrete interaction and data rules in `decisions.md` are WP00 group choices, subject to human group review; they are not additional course mandates.

Version	Date	Reason and change
WP01 draft 1.0	2026-09-30	First SRS baseline derived from the official stories, Lecture 04, and WP00 decisions.
WP05 final source 1.1	2026-09-30	Recorded group 89 and the historical implementation SHA; retained requirement IDs. Python 3.11 verification was pending at that stage.
Group-review candidate 1.2	2026-10-23	Rechecked Lecture 04 structure; added rationale and compatibility evidence.
Independent-review candidate 1.3	2026-10-23	Independently checked official stories and actual code; clarified text/time normalization, ID scope, observable save atomicity and file-error recovery; retained all requirement IDs.

Every numbered requirement below is mandatory within this working specification and uses **shall**; a group-selected rule is not thereby a new course requirement. A verification idea describes how to check a requirement; it is not itself evidence of a passing test. The companion `requirements-catalog.csv` contains the same IDs, statements, source links, and verification ideas for coverage tracking. The current Requirements Coverage and Test Coverage reports record the independently rerun test results and their interpreter versions; this specification does not treat a verification idea or an earlier pass as current evidence.

2. Introduction

2.1 Need and purpose

The PIM lets one user keep notes, tasks, events, and contacts together and retrieve or change them from a local command line. The system supports creating, viewing, modifying, searching, deleting, saving, and loading personal information records (PIRs). It has no required integration with another system; `.pim` files are its interchange boundary between runs.

2.2 Scope and assumptions

The group selected a local Python command-line application. A graphical interface, network service, accounts, database, recurring tasks, tags, and active notifications are outside this baseline. An event's `alarm_time` is stored data; this specification does not require the system to issue an alarm. The official PDF advises that extra features earn no additional credit; the narrow scope is the group's choice.

The user supplies commands and local file paths. Time values represent local wall-clock time at minute precision, without a timezone. A `.pim` file saved by this version can be loaded by this version. No numeric response-time, record-capacity, or coverage threshold is stated by the official PDF or WP00 decisions, so none is invented here.

2.3 Sources and precedence

The official Project Description governs the stories and course constraints. Lecture 04 governs the SRS structure and natural-language conventions. `decisions.md` fixes interaction details left open by the course. If the group changes a choice, it must update that decision and the affected requirement IDs before downstream work. The official requirements matrix records source-page traceability.

3. Glossary

Term	Meaning
PIM	The local Personal Information Management system specified here.
PIR	A personal information record with an ID, a type, and that type's data fields.
Note, task, event, contact	The four canonical PIR types.
Criterion	A search expression that states which PIRs match; it is defined in a `search` command and evaluated during that command.
Text field	`text`, `description`, `name`, `address`, or `mobile_number`.
Time field	`deadline`, `start_time`, or `alarm_time`.
Applicable field	A field present in the schema of the PIR being evaluated.
`.pim`	The exact lowercase file extension for saved PIM data.
Local time	A calendar date and clock time without a timezone or UTC offset.
Failed operation	A command rejected for invalid input or unable to complete because of a file or file I/O error.

4. User requirements definition

4.1 Services requested by users

Story	User service
US1	Create and manage different PIR types in one PIM.
US2	Create a plain-text note.
US3	Create a task with a description and deadline.
US4	Create an event with a description, starting time, and alarm time.
US5	Create a contact with a name, address, and mobile number.
US6	Modify the data of an existing PIR.
US7	Define and execute search criteria over type and stored fields, including text containment, time comparisons, and &&, ||, ! combinations.
US8	Print complete details for one PIR or all PIRs.
US9	Delete a specified PIR.
US10	Save PIRs in a `.pim` file for later use.
US11	Load PIRs from a `.pim` file to continue work.

4.2 User-facing quality and standards

The group chose a single local command-line interface, consistent command grammar, useful error messages, and recoverable invalid-input behavior. The course requires all model code in a separate package named `model`, identifiable other major components, and use of the selected language's standard libraries only. Detailed, checkable constraints appear as NFR-01–NFR-05. The commands and record formats in this document are group choices under the course's permission to decide interaction specifics.

5. System architecture

At a high level, a **command interface** reads commands and presents results; a **PIM core** owns the PIR collection and applies creation, modification, display, deletion, and search rules; and a **file boundary** saves and loads the collection. A command is interpreted by the interface, checked and acted on by the core, and, for `save` or `load`, passed through the file boundary. The core's model code belongs in the required `model` package. The other major components must remain identifiable. This allocation describes responsibilities, not classes, functions, or a fixed implementation design. No reusable architectural component is committed by this SRS.

6. System requirements specification

6.1 Shared record and command conventions

The four schemas and creation argument order are:

Type	Required data fields, in `add` order
`note`	`text`
`task`	`description`, `deadline`
`event`	`description`, `start_time`, `alarm_time`
`contact`	`name`, `address`, `mobile_number`

The public commands are `help`, `add TYPE "FIELD"...`, `list [TYPE]`, `show ID`, `show all`, `update ID FIELD "VALUE"`, `delete ID`, `search EXPR`, `save "PATH.pim"`, `load "PATH.pim"`, and `exit`. Quotation marks indicate a quoted argument, not literal characters in the stored value. One-token non-search values may be unquoted. In quoted arguments, `\"` represents a quote and `\\` represents a backslash. Other backslash escapes are invalid. The entire remainder after `search` is its expression; search literals must be quoted even when they contain one token. Command arguments require separating whitespace; search operators and parentheses may be adjacent to atoms where their boundaries remain clear. Examples in §6.2 illustrate the grammar.

FR-01 [US1] The system shall hold `note`, `task`, `event`, and `contact` PIRs together in one current collection.
Verify: Create one of each type and list the collection.

FR-02 [US2] The system shall create a `note` PIR containing its required `text` field.
Verify: Add a note and show its stored text.

FR-03 [US3] The system shall create a `task` PIR containing its required `description` and `deadline` fields.
Verify: Add a task and show both fields.

FR-04 [US4] The system shall create an `event` PIR containing its required `description`, `start_time`, and `alarm_time` fields without imposing an order between the two times.
Verify: Add an event and show all three fields, including a past time or an alarm after its start.

FR-05 [US5] The system shall create a `contact` PIR containing its required `name`, `address`, and `mobile_number` fields.
Verify: Add a contact and show all three fields, including a number with `+` or a leading zero.

FR-06 [US2–US5] The system shall reject any missing, multiline, or whitespace-only required data field and store each accepted text field after trimming its leading and trailing whitespace; time fields shall follow FR-07 without whitespace normalization.
Verify: Try missing, blank, and multiline values for every schema; verify padded text is trimmed and padded time is rejected without a state change.

FR-07 [US3, US4] The system shall accept time-field values only in valid local `YYYY-MM-DD HH:mm` 24-hour format at minute precision, including valid past times.
Verify: Try valid, impossible-date, wrong-format, and timezone-bearing values; confirm past values are accepted.

FR-08 [US5] The system shall preserve a mobile number as text, including `+` and leading zeros, without region-specific validation.
Verify: Save and show such a number; try a nonempty mobile string without a region-specific pattern.

FR-09 [US1] The system shall initialize an empty session with next ID 1 and assign each successfully created PIR the current next ID, advancing the counter without reusing an ID issued in that session; loading shall preserve the greater counter specified in FR-46.
Verify: Create, delete, and create records in one session; load older and higher-counter snapshots and create again; compare IDs against FR-46.

FR-10 [US1] The system shall leave the next ID unchanged when creation fails.
Verify: Make an invalid `add` between two valid adds and compare assigned IDs.

FR-11 [US1] The system shall allow different PIRs to have equal data-field values.
Verify: Create two records with identical data and observe distinct IDs.

FR-12 [US1–US11] The system shall recognize the command forms listed in §6.1 and reject missing or extra arguments.
Verify: Exercise every form, then remove or add an argument to representative commands.

FR-13 [US1–US11] The system shall match command names, PIR type names, and field names without regard to letter case.
Verify: Repeat representative commands with mixed-case names.

FR-14 [US2–US5] The `add` command shall take data values in the schema order shown in §6.1 and reject an unknown type or a wrong number of fields.
Verify: Add each type in order, then supply an unknown type and too few or too many values.

FR-15 [US1–US11] The command interface shall parse quoted arguments with spaces and the `\"` and `\\` escapes, and reject malformed quoting or any other escape.
Verify: Use spaced and escaped text/path arguments, then unmatched quotes and an unsupported escape.

FR-16 [US6] The `update ID FIELD "VALUE"` command shall change exactly one existing data field of the PIR selected by ID after applying that field's validation rules.
Verify: Update one field of each PIR type; show that other fields are unchanged.

FR-17 [US6] The system shall reject an update to an absent PIR, a field absent from its type, its ID, or its type, without changing the PIR.
Verify: Attempt each invalid update and compare the record before and after.

FR-18 [US1, US8] The `list [TYPE]` command shall show every PIR, or only PIRs of the specified canonical type, in ascending ID order with ID, type, and its complete first text field.
Verify: List a mixed collection with and without a type filter and compare order and full summary text.

FR-19 [US8] The `show ID` command shall display the ID, type, and every data field of the selected PIR.
Verify: Show one PIR of each type and check every field.

FR-20 [US8] The `show all` command shall display the complete details of every current PIR in ascending ID order.
Verify: Create mixed records and compare displayed details and order.

FR-21 [US8] The system shall clearly report an empty collection or an empty `list` type filter instead of presenting a PIR as found.
Verify: Run `list` and `show all` on an empty collection and `list TYPE` with no matching type.

FR-22 [US9] The `delete ID` command shall remove only the specified existing PIR from the current collection.
Verify: Delete one of multiple records and show the remaining records.

FR-23 [US6, US8, US9] The system shall report an absent or invalid ID for `show`, `update`, or `delete` without changing the collection.
Verify: Try zero, negative, noninteger, and missing IDs and compare state.

FR-24 [US1–US11] The `help` command shall display the available command forms and the record, quoting, time, and search syntax needed to use them.
Verify: Compare help output with the command and syntax definitions in this SRS.

FR-25 [US1–US11] The `exit` command or end of input shall terminate the command session normally.
Verify: Run a session ending with `exit` and another ending at EOF.

6.2 Search criteria and execution

A search expression is formed from the atoms below and the operators `&&`, `||`, `!`, and parentheses. A search criterion is **defined** by entering a valid expression after `search`; that same command **executes** it against all current PIRs and displays matches. It is not stored for later commands. The course requires the supported atom categories and logical operators; the exact syntax and evaluation conventions below are WP00 group choices.

The grammar below defines valid grouping. `ATOM` means one of the type, text, or time atoms specified in FR-28, FR-29, and FR-31. Repeated `&&` and `||` operations group left to right.

```
EXPR    := OR_EXPR
OR_EXPR := AND_EXPR ("||" AND_EXPR)*
AND_EXPR:= NOT_EXPR ("&&" NOT_EXPR)*
NOT_EXPR:= "!" NOT_EXPR | "(" EXPR ")" | ATOM
```

FR-26 [US7] The `search EXPR` command shall parse its remaining input as one criterion and reject an empty expression.
Verify: Submit a valid expression, no expression, and a truncated expression.

FR-27 [US7] A valid `search` command shall evaluate its criterion against every PIR in the current collection during that command.
Verify: Create matching and nonmatching PIRs of several types, search, and compare the result set.

FR-28 [US7] The system shall support a type atom `type = "TYPE"` where `TYPE` is one of the four canonical PIR types, matched without regard to case.
Verify: Search for each type with mixed-case type literals; reject an unknown type literal.

FR-29 [US7] The system shall support `FIELD contains "STRING"` for `text`, `description`, `name`, `address`, and `mobile_number`, with a nonempty quoted string.
Verify: Search each of the five text fields; reject empty or unquoted strings.

FR-30 [US7] Text containment shall ignore letter case and match a string occurring anywhere in the applicable field value.
Verify: Search for an interior substring using different case and compare positive and negative matches.

FR-31 [US7] The system shall support `<`, `>`, and `=` comparisons on each of `deadline`, `start_time`, and `alarm_time` against a quoted valid `YYYY-MM-DD HH:mm` value.
Verify: Exercise all nine field-operator pairs, plus invalid and unquoted time literals.

FR-32 [US7] Time comparisons shall compare the represented local date and minute chronologically.
Verify: Test before, after, and equal times across day and month boundaries.

FR-33 [US7] The `&&` operator shall match a PIR only when both operand conditions match it.
Verify: Test all four truth combinations of two atoms.

FR-34 [US7] The `||` operator shall match a PIR when at least one operand condition matches it.
Verify: Test all four truth combinations of two atoms.

FR-35 [US7] The `!` operator shall invert the match result of its following condition.
Verify: Negate matching and nonmatching atoms and a parenthesized expression.

FR-36 [US7] The system shall evaluate parentheses first, then `!`, then `&&`, then `||`.
Verify: Compare expressions whose results distinguish each precedence level and explicit grouping.

FR-37 [US7] A valid atom referring to a field absent from the PIR type being tested shall evaluate false for that PIR, so negating that atom evaluates true.
Verify: Evaluate `deadline` on a note and its negation on the same note.

FR-38 [US7] The system shall reject the entire search for an unknown field, unsupported operator, invalid literal, malformed expression, or unmatched parenthesis without displaying partial matches or changing stored data.
Verify: Submit one case of each error after creating records; compare state and output.

FR-39 [US7] Search results shall be displayed in ascending ID order with each match's ID, type, and complete first text field.
Verify: Create matches in mixed type order and compare the resulting summaries and order.

FR-40 [US7] A valid search with no matches shall clearly report that no PIR matched.
Verify: Search for a valid criterion that matches none of the current PIRs.

Search syntax examples

These illustrate the rules above; they do not add new requirements.

```
search type = "task"
search text contains "meeting"
search description contains "draft"
search name contains "Lee"
search address contains "Kowloon"
search mobile_number contains "0123"
search deadline < "2026-10-01 09:00"
search start_time > "2026-10-01 09:00"
search alarm_time = "2026-10-01 09:00"
search (type = "task" && deadline < "2026-10-01 09:00") || !name contains "Lee"
```

6.3 Persistence and recovery

Version 1 `.pim` files use UTF-8 JSON with exactly the top-level keys `format`, `version`, `next_id`, and `records`. `format` is `COMP3211-PIM`; `version` is the integer `1`. `records` is an ascending-ID array; every entry has `id`, `type`, and exactly the data fields of its type. `next_id` is a positive integer greater than every stored ID. A file is a snapshot: loading it replaces the current collection but retains the higher of the current and file ID counters. This prevents a later creation from reassigning an ID already issued in the current session after an older snapshot is loaded. Across separate sessions, only the counter in a loaded file is retained; no external history of IDs is promised.

FR-41 [US10] The `save "PATH.pim"` command shall store the complete current PIR collection in a file whose extension is exactly `.pim`.
Verify: Save four PIR types, inspect the file path and contents, and reject other extensions.

FR-42 [US10] A saved file shall use the version 1 UTF-8 JSON structure and exact record schemas stated in §6.3.
Verify: Parse a saved file and check encoding, top-level keys/values, IDs, order, and each record's fields.

FR-43 [US10] Saving shall replace the destination with the complete serialized snapshot only after a successful write; if saving fails, the previous destination contents shall remain unchanged.
Verify: Save successfully and compare the complete snapshot; induce serialization, write, and replacement failures and compare the destination's previous bytes.

FR-44 [US11] The `load "PATH.pim"` command shall read a `.pim` file and validate its complete contents before changing current state.
Verify: Load a valid file and separately attempt truncated, wrong-extension, and unreadable files.

FR-45 [US11] Load validation shall reject an incorrect format or version, missing or extra keys or fields, invalid field values, nonpositive or duplicate IDs, or a `next_id` no greater than the maximum stored ID.
Verify: Mutate one aspect of an otherwise valid file for each listed category and attempt load.

FR-46 [US11] Successful loading shall replace, rather than merge with, the current collection and shall use the greater of the pre-load and file `next_id` values for later creation.
Verify: Load a snapshot over different in-memory records, including an older snapshot with a lower counter; add a PIR and inspect the assigned ID.

FR-47 [US11] A failed load shall preserve the entire prior in-memory collection and next ID.
Verify: Record state and the next assigned ID, attempt invalid loads, then compare state and ID assignment.

FR-48 [US10, US11] The system shall resolve a relative `save` or `load` path against the process working directory.
Verify: Run from a chosen directory and use relative paths for save and load.

FR-49 [US1–US11] An unknown command, invalid argument, invalid path or file, or file I/O failure shall produce an error message identifying the rejected input or failure category and return to the command prompt.
Verify: Exercise each error category, inspect the message for the input or category, and issue a subsequent valid command.

FR-50 [US1–US11] A failed command shall not partially change the in-memory collection.
Verify: Compare the collection before and after failed create, update, delete, search, save, and load operations.

The recovery requirement concerns command processing with usable input/output streams. Terminal disconnection or failure of the supplied streams can prevent a prompt from being displayed; it is not classified as a recoverable file operation.

6.4 Non-functional requirements

These are checkable product or implementation constraints, not claims of measured performance. Each has a verification idea. Interpreter versions, environment checks and remaining limits are stated in the current Requirements Coverage report. In particular, an import audit alone does not constitute the disconnected, non-graphical execution check in NFR-01.

NFR-01 [Official p. 1; group scope] The delivered PIM shall operate through a local command-line interface without requiring a GUI or network connection.
Verify: Run the documented commands with network unavailable and no graphical session.

NFR-02 [Group decision] The delivered PIM shall run under Python 3.11 and each newer Python version explicitly listed as supported in the developer manual on its documented platform.
Verify: Run its documented launch and acceptance commands with Python 3.11 and every later version listed as supported.

NFR-03 [Official pp. 1–2] The PIM implementation shall invoke only Python standard-library modules.
Verify: Audit runtime imports and run from an environment without third-party packages.

NFR-04 [Official pp. 1–2] All code belonging to the system model shall reside in a separate package named `model`.
Verify: Inspect the package tree and model behavior exercised by unit tests.

NFR-05 [Official p. 2; group architecture choice] The command-facing and file-facing major components shall be identifiable separately from the `model` package.
Verify: Inspect the delivered source layout and its design document for responsibility mapping.

6.5 US-to-FR traceability

User story	Functional requirements
US1	FR-01, FR-09–FR-11, FR-18
US2	FR-02, FR-06, FR-14
US3	FR-03, FR-06–FR-07, FR-14
US4	FR-04, FR-06–FR-07, FR-14
US5	FR-05–FR-06, FR-08, FR-14
US6	FR-16–FR-17, FR-23
US7	FR-26–FR-40
US8	FR-18–FR-21, FR-23
US9	FR-22–FR-23
US10	FR-41–FR-43, FR-48
US11	FR-44–FR-48

The ranges above are inclusive. FR-12, FR-13, FR-15, FR-24, FR-25, FR-49, and FR-50 apply to **every** story in addition to its row. FR-14 applies to US2–US5. Each story has a direct behavior requirement independent of those cross-cutting rules.

6.6 Rationale for the requirements

Lecture 04 (p. 27) asks for reasons as well as consistent mandatory language. These rationales explain the requirement groups above; they introduce no additional functionality or acceptance criteria. Rules beyond Appendix B remain group decisions for review.

Requirement IDs	Why the requirement is needed
FR-01–FR-05	Users need a single collection while retaining the different information required for notes, tasks, events and contacts.
FR-06–FR-08	Consistent nonempty fields and explicit dates prevent ambiguous records; treating mobile numbers as text preserves leading zeros and international prefixes. Historical events remain useful records.
FR-09–FR-11	Stable identifiers let users select a record reliably after other records are added or removed; rejected creation must not consume an ID.

Requirement IDs	Why the requirement is needed
FR-12–FR-15	Predictable commands, case rules and quoting let users enter spaces and literal characters without guessing how input will be interpreted.
FR-16–FR-17	Editing one field lets users correct information while preserving record identity and unrelated values.
FR-18–FR-25	Summaries support selection; full details support inspection. Explicit empty/not-found messages and help let users distinguish a valid empty result from a failed operation.
FR-26–FR-32	Users need to express type, text and chronological conditions precisely and obtain results over the current collection.
FR-33–FR-40	Boolean composition supports useful combined searches. Explicit precedence, missing-field rules and whole-expression validation prevent surprising matches.
FR-41–FR-43	A saved snapshot must be usable in a later session; a failed save must not destroy the existing saved copy.
FR-44–FR-48	Full-file validation avoids importing partial or inconsistent data. Preserving the ID counter avoids reusing a previously allocated identifier after a load.
FR-49–FR-50	Recoverable errors allow users to correct input without restarting or losing valid records.
NFR-01–NFR-03	A local CLI and standard-library implementation keep setup appropriate to the course scope. The explicit Python support list makes compatibility verifiable.
NFR-04–NFR-05	Keeping the model separate enables direct unit tests, while identifiable command and file components make responsibilities reviewable.